Provider 제공자

개요

Provider(제공자) 시스템은 IPFS 네트워크에서 노드가 보유한 콘텐츠를 다른 노드들에게 알리는 역할을 담당합니다. Boxo의 provider 모듈은 콘텐츠 광고(advertising), 재제공(reproviding), 그리고 효율적인 배치 처리를 통해 네트워크에서 데이터의 가용성을 보장합니다.

주요 개념

Provider의 역할

1. **콘텐츠 광고**: 노드가 특정 CID를 보유하고 있음을 네트워크에 알림
2. **재제공**: 주기적으로 콘텐츠 광고를 갱신하여 가용성 유지
3. **배치 처리**: 여러 CID를 효율적으로 한 번에 처리

제공자 시스템 구성 요소

- **Provider**: 개별 CID를 네트워크에 광고
- **Reprovider**: 주기적으로 콘텐츠를 재광고
- **KeyChanFunc**: CID 스트림을 제공하는 함수
- **System**: 전체 제공자 시스템을 관리

기본 사용법

Provider 시스템 설정

```go
package main


import (
"context"
"log"
"time"

"github.com/ipfs/boxo/provider"
"github.com/ipfs/go-cid"
"github.com/ipfs/go-datastore"
"github.com/ipfs/go-datastore/sync"
)

func main() {
ctx := context.Background()
// 데이터스토어 설정
dstore := sync.MutexWrap(datastore.NewMapDatastore())
// 기본 Provider 시스템 생성 (오프라인 모드)
providerSystem, err := provider.New(dstore)
if err != nil {
log.Fatal(err)
}
```

```
defer  providerSystem.Close()

// CID 제공 요청

targetCid := cid.MustParse("QmYourContent...")

err = providerSystem.Provide(ctx, targetCid, true)

if  err != nil {

log.Fatal(err)

}

log.Printf("CID %s가 제공 대기열에 추가되었습니다", targetCid)

}
```

온라인 Provider 설정

```
// 실제 네트워크에 광고하는 Provider 설정

type mockContentRouter struct{}


func (m *mockContentRouter) Provide(ctx context.Context, c cid.Cid, announce  bo

log.Printf("네트워크에 CID %s 광고 중...", c)

// 실제 DHT나 다른 라우팅 시스템에 광고

return  nil

}


func  setupOnlineProvider() {

dstore := sync.MutexWrap(datastore.NewMapDatastore())
```

```
router := &mockContentRouter{}

// 온라인 Provider 시스템 생성

providerSystem, err := provider.New(dstore,

provider.Online(router),

provider.ReproviderInterval(24*time.Hour), // 24시간마다 재제공

provider.ProvideWorkerCount(32), // 32개 워커 사용

)

if err != nil {

log.Fatal(err)

}

defer providerSystem.Close()

log.Println("온라인 Provider 시스템이 시작되었습니다")

}
```

고급 활용

키 제공자 설정

```
// 핀된 콘텐츠를 제공하는 KeyChanFunc

func createPinnedKeyProvider(pinner pin.Pinner) provider.KeyChanFunc {

return func(ctx context.Context) (<-chan cid.Cid, error) {

outCh := make(chan cid.Cid)

go func() {
```

```
defer close(outCh)

// 재귀적으로 핀된 CID들 수집

recursivePins := pinner.RecursiveKeys(ctx, false)

for streamedPin := range recursivePins {

if streamedPin.Err != nil {

log.Printf("핀 조회 오류: %v", streamedPin.Err)

continue

}

select {

case outCh <- streamedPin.Pin.Key:

case <-ctx.Done():

return

}

}

// 직접 핀된 CID들 수집

directPins := pinner.DirectKeys(ctx, false)

for streamedPin := range directPins {

if streamedPin.Err != nil {

log.Printf("핀 조회 오류: %v", streamedPin.Err)

continue

}

select {

case outCh <- streamedPin.Pin.Key:
```

```
case <-ctx.Done():
return
}
}
}()
return  outCh, nil
}
}


// 키 제공자를 Provider 시스템에 설정
func  setupWithKeyProvider(pinner pin.Pinner) {
dstore := sync.MutexWrap(datastore.NewMapDatastore())
keyProvider := createPinnedKeyProvider(pinner)
providerSystem, err := provider.New(dstore,
provider.KeyProvider(keyProvider),
provider.ReproviderInterval(12*time.Hour),
)
if  err != nil {
log.Fatal(err)
}
// 키 제공자 동적 변경
newKeyProvider := func(ctx context.Context) (<-chan cid.Cid, error) {
// 새로운 키 제공 로직
```

```
ch := make(chan cid.Cid)

close(ch)

return ch, nil

}

providerSystem.SetKeyProvider(newKeyProvider)

}
```

DAG Provider 사용

```
// 특정 DAG의 모든 블록을 제공

func setupDAGProvider(rootCid cid.Cid, fetcherFactory fetcher.Factory) {

// DAG Provider 생성

dagProvider := provider.NewDAGProvider(rootCid, fetcherFactory)

dstore := sync.MutexWrap(datastore.NewMapDatastore())

providerSystem, err := provider.New(dstore,

provider.KeyProvider(dagProvider),

provider.ReproviderInterval(6*time.Hour),

)

if err != nil {

log.Fatal(err)

}

defer providerSystem.Close()

log.Printf("DAG %s의 모든 블록이 제공됩니다", rootCid)
```

```
}
```

우선순위 기반 Provider

```
// 우선순위가 있는 CID 제공
func setupPrioritizedProvider() {
// 높은 우선순위 CID 제공자
priorityCids := func(ctx context.Context) (<-chan cid.Cid, error) {
ch := make(chan cid.Cid, 10)
defer close(ch)
// 중요한 CID들을 먼저 제공
importantCids := []cid.Cid{
cid.MustParse("QmImportant1..."),
cid.MustParse("QmImportant2..."),
}
for _, c := range importantCids {
select {
case ch <- c:
case <-ctx.Done():
return ch, ctx.Err()
}
}
return ch, nil
```

```
}

// 일반 우선순위 CID 제공자

otherCids := func(ctx context.Context) (<-chan cid.Cid, error) {

ch := make(chan cid.Cid, 100)

defer  close(ch)

// 일반 CID들 제공

// ... 일반 CID 로직

return  ch, nil

}

// 우선순위 기반 제공자 생성

prioritizedProvider := provider.NewPrioritizedProvider(priorityCids, otherCids)

dstore := sync.MutexWrap(datastore.NewMapDatastore())

providerSystem, err := provider.New(dstore,

provider.KeyProvider(prioritizedProvider),

)

if  err != nil {

log.Fatal(err)

}

defer  providerSystem.Close()

}
```

버퍼링된 Provider

```go
// 메모리 버퍼링을 사용한 효율적인 Provider
func setupBufferedProvider() {
// 기본 키 제공자
baseKeyProvider := func(ctx context.Context) (<-chan cid.Cid, error) {
ch := make(chan cid.Cid)
go func() {
defer close(ch)
// 대량의 CID를 천천히 생성
for i := 0; i < 10000; i++ {
// 시뮬레이션: 느린 CID 생성
time.Sleep(time.Millisecond)
// 가상의 CID 생성
c := cid.MustParse(fmt.Sprintf("QmExample%d", i))
select {
case ch <- c:
case <-ctx.Done():
return
}
}
}()
return ch, nil
}
```

```
// 버퍼링된 제공자로 래핑
bufferedProvider := provider.NewBufferedProvider(baseKeyProvider)
dstore := sync.MutexWrap(datastore.NewMapDatastore())
providerSystem, err := provider.New(dstore,
provider.KeyProvider(bufferedProvider),
provider.MaxBatchSize(1000), // 배치 크기 제한
)
if err != nil {
log.Fatal(err)
}
defer providerSystem.Close()
}
```

제공자 등록과 광고

수동 제공 요청

```
// 개별 CID 제공
func provideSingleCID(providerSystem provider.System, c cid.Cid) error {
ctx := context.Background()
err := providerSystem.Provide(ctx, c, true) // true: 광고 활성화
if err != nil {
return fmt.Errorf("CID %s 제공 실패: %w", c, err)
```

```
}

log.Printf("CID %s가 제공 대기열에 추가되었습니다", c)

return nil

}



// 여러 CID 배치 제공

func provideBatchCIDs(providerSystem provider.System, cids []cid.Cid) error {

ctx := context.Background()

for i, c := range cids {

err := providerSystem.Provide(ctx, c, true)

if err != nil {

log.Printf("CID %s 제공 실패: %v", c, err)

continue

}

// 진행 상황 로깅

if (i+1)%100 == 0 {

log.Printf("진행 상황: %d/%d CID 처리됨", i+1, len(cids))

}

}

return nil

}
```

재제공 전략

```go
// 수동 재제공 실행
func manualReprovide(providerSystem provider.System) error {
ctx := context.Background()
log.Println("재제공 시작...")
start := time.Now()
err := providerSystem.Reprovide(ctx)
if err != nil {
return fmt.Errorf("재제공 실패: %w", err)
}
duration := time.Since(start)
log.Printf("재제공 완료 (소요 시간: %v)", duration)
return nil
}


// 조건부 재제공
func conditionalReprovide(providerSystem provider.System) {
ticker := time.NewTicker(1 * time.Hour)
defer ticker.Stop()
for {
select {
case <-ticker.C:
stats, err := providerSystem.Stat()
```

```
if err != nil {
log.Printf("통계 조회 실패: %v", err)
continue
}
// 마지막 재제공으로부터 12시간이 지났으면 재제공
if time.Since(stats.LastRun) > 12*time.Hour {
log.Println("조건부 재제공 실행...")
err := providerSystem.Reprovide(context.Background())
if err != nil {
log.Printf("재제공 실패: %v", err)
}
}
}
}
}
```

성능 최적화

처리량 모니터링

```
// 처리량 콜백 설정
func setupThroughputMonitoring() {
throughputCallback := func(reprovide, complete bool, totalKeysProvided uint, t
```

```
rate := float64(totalKeysProvided) / totalDuration.Seconds()

if reprovide {

log.Printf("재제공 처리량: %.2f CID/초 (총 %d CID, %v 소요)",

rate, totalKeysProvided, totalDuration)

} else {

log.Printf("제공 처리량: %.2f CID/초 (총 %d CID, %v 소요)",

rate, totalKeysProvided, totalDuration)

}

if complete {

log.Println("모든 CID 처리 완료")

}

return true // 계속 모니터링

}

dstore := sync.MutexWrap(datastore.NewMapDatastore())

providerSystem, err := provider.New(dstore,

provider.ThroughputReport(throughputCallback, 100), // 100개마다 보고

)

if err != nil {

log.Fatal(err)

}

defer providerSystem.Close()

}
```

워커 수 최적화

```
// 시스템 리소스에 따른 워커 수 조정
func optimizeWorkerCount() {
// CPU 코어 수 기반 워커 수 설정
workerCount := runtime.NumCPU() * 2
dstore := sync.MutexWrap(datastore.NewMapDatastore())
providerSystem, err := provider.New(dstore,
provider.ProvideWorkerCount(workerCount),
provider.MaxBatchSize(500), // 메모리 사용량 제한
)
if err != nil {
log.Fatal(err)
}
defer providerSystem.Close()
log.Printf("워커 수: %d, 최대 배치 크기: 500", workerCount)
}

// 무제한 워커 사용 (고성능 시나리오)
func setupUnlimitedWorkers() {
dstore := sync.MutexWrap(datastore.NewMapDatastore())
providerSystem, err := provider.New(dstore,
provider.ProvideWorkerCount(0), // 0 = 무제한
```

```
)
if err != nil {
log.Fatal(err)
}
defer providerSystem.Close()
log.Println("무제한 워커로 Provider 시스템 시작")
}
```

배치 크기 조정

```
// 메모리 사용량에 따른 배치 크기 조정
func adaptiveBatchSize() {
var m runtime.MemStats
runtime.ReadMemStats(&m)
// 사용 가능한 메모리에 따라 배치 크기 결정
availableMemMB := (m.Sys - m.Alloc) / 1024 / 1024
batchSize := uint(availableMemMB / 10) // 10MB당 1개 CID
if batchSize < 100 {
batchSize = 100
} else if batchSize > 10000 {
batchSize = 10000
}
dstore := sync.MutexWrap(datastore.NewMapDatastore())
```

```
providerSystem, err := provider.New(dstore,
provider.MaxBatchSize(batchSize),
)
if err != nil {
log.Fatal(err)
}
defer providerSystem.Close()
log.Printf("적응형 배치 크기: %d", batchSize)
}
```

모니터링 및 통계

Provider 통계 조회

```
// Provider 시스템 통계 모니터링
func monitorProviderStats(providerSystem provider.System) {
ticker := time.NewTicker(5 * time.Minute)
defer ticker.Stop()
for {
select {
case <-ticker.C:
stats, err := providerSystem.Stat()
if err != nil {
```

```
log.Printf("통계 조회 실패: %v", err)

continue

}

log.Printf("Provider 통계:")

log.Printf(" 총 재제공 수: %d", stats.TotalReprovides)

log.Printf(" 마지막 배치 크기: %d", stats.LastReprovideBatchSize)

log.Printf(" 재제공 간격: %v", stats.ReprovideInterval)

log.Printf(" 평균 재제공 시간: %v", stats.AvgReprovideDuration)

log.Printf(" 마지막 재제공 시간: %v", stats.LastReprovideDuration)

log.Printf(" 마지막 실행: %v", stats.LastRun)

}

}

}
```

대기열 관리

```
// 제공 대기열 상태 관리

func manageProvideQueue(providerSystem provider.System) {

// 대기열 크기 모니터링

go func() {

ticker := time.NewTicker(1 * time.Minute)

defer ticker.Stop()

for {
```

```
select {
case <-ticker.C:
// 대기열이 너무 크면 경고
// (실제 구현에서는 대기열 크기 조회 API 필요)
log.Println("대기열 상태 확인 중...")
}
}
}()
// 필요시 대기열 정리
clearQueue := func() {
cleared := providerSystem.Clear()
log.Printf("대기열에서 %d개 항목 제거됨", cleared)
}
// 예시: 특정 조건에서 대기열 정리
_ = clearQueue
}
```

모범 사례

1. 효율적인 키 제공자 구현

```
// 캐시를 활용한 효율적인 키 제공자
type cachedKeyProvider struct {
```

```
cache []cid.Cid

cacheTime time.Time

cacheTTL time.Duration

baseProvider provider.KeyChanFunc

}



func (c *cachedKeyProvider) provide(ctx context.Context) (<-chan cid.Cid, error)

// 캐시가 유효하면 캐시된 결과 사용

if time.Since(c.cacheTime) < c.cacheTTL && len(c.cache) > 0 {

ch := make(chan cid.Cid, len(c.cache))

for _, cid := range c.cache {

ch <- cid

}

close(ch)

return ch, nil

}

// 캐시 갱신

baseCh, err := c.baseProvider(ctx)

if err != nil {

return nil, err

}

outCh := make(chan cid.Cid)

go func() {
```

```
defer close(outCh)

var newCache []cid.Cid

for cid := range baseCh {

newCache = append(newCache, cid)

select {

case outCh <- cid:

case <-ctx.Done():

return

}

}

c.cache = newCache

c.cacheTime = time.Now()

}()

return outCh, nil

}
```

2. 오류 처리 및 복구

```
// 견고한 Provider 시스템 구현

func robustProviderSystem() {

dstore := sync.MutexWrap(datastore.NewMapDatastore())

// 재시도 로직이 있는 라우터

type retryRouter struct {
```

```
base provider.Provide

retries  int

}

router := &retryRouter{

base: &mockContentRouter{},

retries: 3,

}

providerSystem, err := provider.New(dstore,

provider.Online(router),

provider.ReproviderInterval(24*time.Hour),

)

if  err != nil {

log.Fatal(err)

}

defer  providerSystem.Close()

// 주기적 건강성 검사

go  func() {

ticker := time.NewTicker(10 * time.Minute)

defer  ticker.Stop()

for {

select {

case <-ticker.C:

// 시스템 상태 확인
```

```
_, err := providerSystem.Stat()
if err != nil {
log.Printf("Provider 시스템 오류: %v", err)
// 필요시 시스템 재시작 로직
}
}
}
}()
}
```

3. 리소스 관리

```
// 메모리 효율적인 Provider 설정
func memoryEfficientProvider() {
dstore := sync.MutexWrap(datastore.NewMapDatastore())
// 작은 배치 크기로 메모리 사용량 제한
providerSystem, err := provider.New(dstore,
provider.MaxBatchSize(100),
provider.ProvideWorkerCount(4), // 적은 워커 수
)
if err != nil {
log.Fatal(err)
}
```

```
defer providerSystem.Close()

// 메모리 사용량 모니터링

go func() {

ticker := time.NewTicker(1 * time.Minute)

defer ticker.Stop()

for {

select {

case <-ticker.C:

var m runtime.MemStats

runtime.ReadMemStats(&m)

if m.Alloc > 100*1024*1024 { // 100MB 초과시

log.Println("메모리 사용량이 높습니다. GC 실행...")

runtime.GC()

}

}

}

}()

}
```

문제 해결

일반적인 문제와 해결책

1. **제공 속도가 느림**

```
// 워커 수 증가
provider.ProvideWorkerCount(64)
// 배치 크기 증가
provider.MaxBatchSize(1000)
```

2. **메모리 사용량 과다**

```
// 배치 크기 감소
provider.MaxBatchSize(100)
// 버퍼링 비활성화
// NewBufferedProvider 사용하지 않음
```

3. **재제공이 실행되지 않음**

```
// 재제공 간격 확인
stats, _ := providerSystem.Stat()
log.Printf("재제공 간격: %v", stats.ReprovideInterval)
// 수동 재제공 실행
providerSystem.Reprovide(context.Background())
```

Provider 시스템은 IPFS 네트워크에서 콘텐츠 가용성을 보장하는 핵심 구성 요소입니다. 적절한 설정과 모니터링을 통해 효율적이고 안정적인 콘텐츠 제공 서비스를 구축할 수 있습니다.